# Homework 4: function approximation and Monte Carlo methods

## Submission

Due: Oct 26, 2026

Submit the following three files in your directory:

1. `lastname_firstname_hw04.ipynb`: one executed Jupyter notebook containing all code and markdown text;
2. `lastname_firstname_hw04.pdf`: a PDF compiled or exported from the executed notebook; and
3. `requirements.txt`: a short list of the packages needed to run the notebook, with version numbers.

Your notebook must run from top to bottom in a fresh session without manual edits. Use the supplied random seeds and NumPy/SciPy routines rather than implementing linear algebra algorithms from scratch. Report requested numerical results with at least three significant digits. Keep explanations concise, but interpret the numerical behavior rather than reporting outputs alone.

## Question 1: Basis choice and approximation error (50 points)

Generate noisy observations from a smooth curve with both periodic and localized structure:

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.polynomial.legendre import legvander
from scipy.interpolate import BSpline

rng = np.random.default_rng(2026)
n = 120
x = np.sort(rng.uniform(0, 1, n))

def f_true(x):
    return np.sin(2 * np.pi * x) + 0.65 * np.exp(-160 * (x - 0.72)**2)

y = f_true(x) + 0.25 * rng.normal(size=n)
x_grid = np.linspace(0, 1, 500)
```

Use the following functions to construct three bases with the same number $J$ of columns:

```
def polynomial_basis(x, J):
    return legvander(2 * np.asarray(x) - 1, J - 1)

def fourier_basis(x, J):
    x = np.asarray(x)
    m = (J - 1) // 2
    columns = [np.ones_like(x)]
    for k in range(1, m + 1):
        columns += [np.cos(2 * np.pi * k * x), np.sin(2 * np.pi * k * x)]
    return np.column_stack(columns)

def bspline_basis(x, J, degree=3):
    K = J - degree - 1
```

```
    interior = np.linspace(0, 1, K + 2)[1:-1]
    knots = np.r_[np.repeat(0.0, degree + 1), interior,
                  np.repeat(1.0, degree + 1)]
    return BSpline.design_matrix(x, knots, degree).toarray()
```

1. Using $J = 11$, construct the Legendre-polynomial, Fourier, and cubic B-spline design matrices. Report each matrix's dimensions and condition number $\kappa_2(\Phi)$. Explain why the predictor is rescaled to $[-1, 1]$ before evaluating the Legendre basis.
2. Fit each model by least squares. In one figure, show the observations, the true curve, and the three fitted curves. For each fit, report the training RMSE and the true-curve grid RMSE

$$\left[\frac{1}{500}\sum_{r=1}^{500}\{\hat{f}(x_r) - f_{\text{true}}(x_r)\}^2\right]^{1/2},$$

   where $\{x_r\}$ is a grid over the support of [0, 1].
3. Repeat the fits for $J \in \{7, 11, 15\}$. Present a table containing the training RMSE and true-curve grid RMSE for every basis and value of $J$. Explain why increasing $J$ cannot increase training error but can increase the true-curve grid error.

## Question 2: P-splines and selection of smoothness (50 points)

Use the same `x`, `y`, `x_grid`, and `f_true` from Question 1. Construct a rich cubic B-spline design with $J = 30$ basis functions:

```
J = 30
Phi = bspline_basis(x, J)
Phi_grid = bspline_basis(x_grid, J)
D2 = np.diff(np.eye(J), n=2, axis=0)
Omega = D2.T @ D2
```

For a given $\lambda \geq 0$, the P-spline estimate satisfies

$$(\Phi^\top\Phi + \lambda\Omega)\hat{\beta}_\lambda = \Phi^\top y.$$

1. For 100 logarithmically spaced values from $10^{-6}$ to $10^6$, compute

$$S_\lambda = \Phi(\Phi^\top\Phi + \lambda\Omega)^{-1}\Phi^\top, \qquad \mathrm{df}(\lambda) = \mathrm{tr}(S_\lambda),$$

   and

$$\mathrm{GCV}(\lambda) = \frac{n^{-1}\|y - S_\lambda y\|_2^2}{\{1 - \mathrm{df}(\lambda)/n\}^2}.$$

   Select the $\lambda$ that minimizes GCV. Report the selected $\lambda$, its effective degrees of freedom, and its GCV value. Produce a two-panel figure showing GCV and effective degrees of freedom against $\lambda$ on a logarithmic horizontal axis.
2. Compare the unpenalized fit, the GCV-selected fit, and an over-smoothed fit with $\lambda = 10^6$. Plot all three with the observations and true curve.

## Grading guide

| Criterion | Points |
|---|---|
| Correct implementation and numerical results | 50 |

| Criterion | Points |
|---|---|
| Sound analysis and interpretation | 25 |
| Clear figures, tables, and concise communication | 15 |
| Reproducibility and complete submission | 10 |
| **Total** | **100** |

Minor differences caused by software versions and timing variability are acceptable. Numerical results without written interpretation will receive limited credit.